FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	**Kelzo Marcel O. Lumabas**
Section:	**AN42**
Professor:	**Ms. Crisola G. Tan**
Date:	**October 5, 2026**

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)

- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

```
# ============================================================
# KNOWLEDGE REPRESENTATION, RULE-BASED REASONING,
# AND CASE-BASED REASONING
# ============================================================

# ============================================================
# PART 1 - KNOWLEDGE BASE
# ============================================================

facts = {
    "temperature": 32,
    "is_raining": True,
    "humidity": 85,
    "person_present": True,
    "fan_on": False,
    "window_open": False
}

# ============================================================
# PART 2 - RULE-BASED REASONING
# ============================================================

# Rule 1
def rule_hot_room(facts):

    if facts["temperature"] > 30 and facts["person_present"]:
        return "Turn on the fan"

# Rule 2
def rule_raining(facts):

    if facts["is_raining"] and facts["window_open"]:
        return "Close the window"

# Rule 3
def rule_high_humidity(facts):

    if facts["humidity"] > 80 and facts["person_present"]:
        return "Turn on the fan"
```

```python
# Inference engine
def inference_engine(facts):

    rules = [
        rule_hot_room,
        rule_raining,
        rule_high_humidity
    ]

    actions = []

    for rule in rules:

        result = rule(facts)

        if result is not None:
            actions.append(result)

    # Remove duplicate actions
    return list(set(actions))

# Display RBR results
print("========================================")
print("RULE-BASED REASONING")
print("========================================")

actions = inference_engine(facts)

print("\nKnowledge Base:")

for key, value in facts.items():
    print(f"{key}: {value}")

print("\nRule-Based Reasoning Actions:")

for action in actions:
    print("-", action)

# ============================================================
# PART 3 - CASE-BASED REASONING
# ============================================================

case_base = [
```

```
    {
        "problem": {
            "temperature": 32,
            "is_raining": True,
            "humidity": 85,
            "person_present": True
        },
        "solution": "Turn on the fan and close the window."
    },

    {
        "problem": {
            "temperature": 25,
            "is_raining": False,
            "humidity": 50,
            "person_present": True
        },
        "solution": "Keep the fan off and open the window."
    },

    {
        "problem": {
            "temperature": 35,
            "is_raining": False,
            "humidity": 70,
            "person_present": True
        },
        "solution": "Turn on the fan."
    }
]

# Calculate similarity
def calculate_similarity(current_problem, past_problem):

    score = 0

    for feature in current_problem:

        if current_problem[feature] == past_problem[feature]:
            score += 1

    return score
```

```
# Retrieve most similar case
def retrieve_case(current_problem, case_base):

    best_case = None
    best_score = -1

    for case in case_base:

        score = calculate_similarity(
            current_problem,
            case["problem"]
        )

        if score > best_score:

            best_score = score
            best_case = case

    return best_case, best_score

# Revise solution
def revise_solution(solution):

    print("\nRetrieved solution:")
    print(solution)

    choice = input(
        "\nDo you want to keep this solution? (y/n): "
    ).lower()

    if choice == "y":

        return solution

    else:

        new_solution = input(
            "Enter the revised solution: "
        )

        return new_solution
```

```python
# Retain new case
def retain_case(problem, solution, case_base):

    new_case = {
        "problem": problem.copy(),
        "solution": solution
    }

    case_base.append(new_case)

    print("\nNew case has been retained.")

# ============================================================
# CBR SIMULATION
# ============================================================

print("\n==========================================")
print("CASE-BASED REASONING")
print("==========================================")

current_problem = {
    "temperature": facts["temperature"],
    "is_raining": facts["is_raining"],
    "humidity": facts["humidity"],
    "person_present": facts["person_present"]
}

# RETRIEVE
best_case, score = retrieve_case(
    current_problem,
    case_base
)

print("\nMost similar past case:")
print(best_case["problem"])

print("\nSimilarity score:", score)

# REUSE
solution = best_case["solution"]

# REVISE
solution = revise_solution(solution)
```

```
# RETAIN
retain_case(
    current_problem,
    solution,
    case_base
)

print("\nFinal CBR solution:")
print(solution)

print("\nTotal cases in case base:", len(case_base))
```

1. Rule-Based Reasoning (RBR) was easier to implement since it only requires simple IF–THEN rules. The programmer directly defines the conditions and the actions that should happen when those conditions are met. Case-Based Reasoning (CBR) requires additional steps such as storing past cases, comparing the current problem with previous cases, retrieving the most similar case, and revising and retaining the solution.
2. CBR is more flexible when solving unstructured problems since it can use previous experiences to handle situations that may not have an exact predefined rule. Instead of requiring a rule for every possible situation, the system can find a similar previous case and adapt its solution. RBR is more predictable but depends heavily on the rules created by the programmer.
3. For situations with continuously changing data, RBR can be better when the changes can be represented by clear conditions and thresholds, because the system can immediately check the current facts against its rules. However, CBR can also adapt over time by retaining new cases and solutions. The better approach therefore depends on the type of changing information and the requirements of the system.
4. RBR is commonly used in expert systems, medical decision support, troubleshooting systems, and automated control systems. For example, a smart home system can turn on an air conditioner when the temperature becomes too high. On the other hand, CBR can be used in customer support, medical diagnosis, equipment troubleshooting, and recommendation systems, where previous problems and their solutions can help solve new problems.